# Laravel Admin Project Reference

This document is a Laravel-only reference for building a practical admin task project from scratch, starting from authentication, admin layout, CRUD module structure, and server-side DataTables integration.

It is designed as a reusable reference for practical interview assignments where a company gives a Laravel admin task and expects a clean, working, easy-to-explain project.

## Project Purpose

This project structure is useful when the task asks for:

- Laravel admin login
- protected admin panel
- CRUD module
- reusable create/edit form
- searchable and paginated listing
- simple but secure enough structure

A shared Blade partial for forms is a common Laravel reuse pattern, and Yajra DataTables is a standard package for server-side searchable admin listings.[cite:147] [cite:121]

## Recommended Stack

- Laravel 11
- Blade templates
- Bootstrap/AdminLTE style admin panel
- MySQL
- Yajra DataTables
- Eloquent ORM

Laravel Collective HTML is not recommended for Laravel 11 because the package is abandoned, so plain Blade forms are the safer approach.[cite:140] [cite:146]

## Initial Project Setup

### 1. Create Laravel project

```
composer create-project laravel/laravel delta-backend
cd delta-backend
```

### 2. Configure environment

Update `.env` database values:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=delta_backend
DB_USERNAME=root
DB_PASSWORD=
```

### 3. Run migration

```
php artisan migrate
```

## Authentication Setup

A protected admin panel needs authentication before any module is created. Laravel starter authentication is the fastest clean setup for practical tasks.

### 4. Install authentication starter

Use Laravel Breeze for a quick auth scaffold:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
npm run build
php artisan migrate
```

Laravel starter kits are intended to quickly provide login, registration, password reset, and session-based auth flows for Blade applications.[cite:177]

### 5. Basic auth flow

After Breeze installation, these are available:

- login
- register
- forgot password
- dashboard route

- auth middleware

For practical tasks, registration can later be disabled if only admin login is needed.

### Admin Panel Structure

After auth is working, create a separate admin area protected by middleware.

## 6. Suggested Laravel file structure

```
app/
├── Http/
│   └── Controllers/
│       └── Admin/
│           ├── DashboardController.php
│           └── PageController.php
├── Models/
│   ├── User.php
│   └── Page.php

resources/views/
├── admin/
│   ├── layouts/
│   │   └── admin.blade.php
│   ├── partials/
│   │   ├── navbar.blade.php
│   │   └── sidebar.blade.php
│   ├── dashboard/
│   │   └── index.blade.php
│   └── pages/
│       ├── index.blade.php
│       ├── create.blade.php
│       ├── edit.blade.php
│       └── _form.blade.php
└── auth/

routes/
├── web.php
└── api.php

database/
├── migrations/
├── factories/
└── seeders/
```

### Admin Routes

### 7. Create admin routes

In `routes/web.php`:

```
use App\Http\Controllers\Admin\DashboardController;
use App\Http\Controllers\Admin\PageController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('login');
});

Route::middleware(['auth'])->prefix('admin')->name('admin.')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard

    Route::get('pages-datatable', [PageController::class, 'datatable'])->name('pages
    Route::resource('pages', PageController::class);
});

require __DIR__.'/auth.php';
```

Route groups with auth middleware and name prefixes are a standard Laravel way to organize protected admin routes.[cite:153] [cite:160]

## Dashboard Setup

### 8. Create dashboard controller

Create file:

`app/Http/Controllers/Admin/DashboardController.php`

```
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;

class DashboardController extends Controller
{
    public function index()
    {
        return view('admin.dashboard.index');
    }
}
```

### 9. Create admin layout

Create file:

`resources/views/admin/layouts/admin.blade.php`

This layout should contain:

- sidebar
- top navbar
- logout button
- content section
- jQuery
- Bootstrap JS
- DataTables CSS/JS
- `@stack('scripts')`

The correct script order is important because DataTables depends on jQuery being loaded first.[cite:162] [cite:169]

Suggested script order at bottom of layout:

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.bundle.
<script src="https://cdn.datatables.net/1.13.8/js/jquery.dataTables.min.js"></script
<script src="https://cdn.datatables.net/1.13.8/js/dataTables.bootstrap4.min.js"></sc
@stack('scripts')
```

### First CRUD Module: Pages

The first module created was `Pages`, which serves as a reusable pattern for other modules like Blog Posts, Categories, Videos, Services, and Leads.

### 10. Create model and migration

```
php artisan make:model Page -m
```

### 11. Pages migration example

Fields used:

- title
- slug
- meta_title
- meta_keywords

- meta_description
- canonical_url
- og_title
- og_description
- og_image
- schema_json
- sort_order
- status
- is_published
- published_at

Example migration:

```
Schema::create('pages', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('slug')->unique();
    $table->boolean('status')->default(1);
    $table->boolean('is_published')->default(1);
    $table->integer('sort_order')->default(0);
    $table->timestamp('published_at')->nullable();
    $table->string('meta_title')->nullable();
    $table->text('meta_keywords')->nullable();
    $table->text('meta_description')->nullable();
    $table->string('canonical_url')->nullable();
    $table->string('og_title')->nullable();
    $table->text('og_description')->nullable();
    $table->string('og_image')->nullable();
    $table->longText('schema_json')->nullable();
    $table->timestamps();
});
```

### 12. Run migration

```
php artisan migrate
```

### 13. Create Page model

`app/Models/Page.php`

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
```

```
class Page extends Model
{
    protected $fillable = [
        'title',
        'slug',
        'status',
        'is_published',
        'sort_order',
        'published_at',
        'meta_title',
        'meta_keywords',
        'meta_description',
        'canonical_url',
        'og_title',
        'og_description',
        'og_image',
        'schema_json',
    ];

    protected $casts = [
        'status' => 'boolean',
        'is_published' => 'boolean',
        'published_at' => 'datetime',
    ];
}
```

## Pages Controller

### 14. Create controller

```
php artisan make:controller Admin/PageController --resource
```

### 15. Controller methods used

- `index()`
- `datatable()`
- `create()`
- `store()`
- `edit()`
- `update()`
- `destroy()`

The `datatable()` method is extra and serves AJAX JSON for Yajra DataTables.[cite:121] [cite:136]

## DataTables Installation

### 16. Install package

Use the package version that matches the project environment.

```
composer require yajra/laravel-datatables
```

Yajra DataTables is the common Laravel integration package for jQuery DataTables with Eloquent and server-side processing.[cite:121] [cite:124]

### 17. Datatable controller method

```
use App\Models\Page;
use Illuminate\Http\Request;
use Yajra\DataTables\Facades\DataTables;

public function datatable(Request $request)
{
    $query = Page::select([
        'id',
        'title',
        'slug',
        'meta_title',
        'status',
        'is_published',
        'published_at',
        'created_at'
    ]);

    return DataTables::of($query)
        ->addIndexColumn()
        ->editColumn('status', function ($page) {
            return $page->status
                ? '<span class="badge-soft-success">Active</span>'
                : '<span class="badge-soft-danger">Inactive</span>';
        })
        ->editColumn('is_published', function ($page) {
            return $page->is_published
                ? '<span class="badge-soft-primary">Published</span>'
                : '<span class="badge-soft-warning">Draft</span>';
        })
        ->editColumn('published_at', function ($page) {
            return $page->published_at
                ? \Carbon\Carbon::parse($page->published_at)->format('d M Y h:i A')
                : '-';
        })
        ->addColumn('action', function ($page) {
            $editUrl = route('admin.pages.edit', $page->id);
            $deleteUrl = route('admin.pages.destroy', $page->id);

            return '
                <a href="'.$editUrl.'" class="btn btn-warning btn-sm">
```

```
                    <i class="fas fa-edit"></i>
                </a>
                <form action="'.$deleteUrl.'" method="POST" class="d-inline-block" c
                    '.csrf_field().'
                    '.method_field('DELETE').'
                    <button type="submit" class="btn btn-danger btn-sm">
                        <i class="fas fa-trash"></i>
                    </button>
                </form>
            ';
        })
        ->rawColumns(['status', 'is_published', 'action'])
        ->make(true);
}
```

HTML columns in Yajra need to be marked with `rawColumns()` so the badge and action buttons render correctly.[cite:133] [cite:136]

## Pages Views Structure

### 18. Create view files

```
resources/views/admin/pages/
├── index.blade.php
├── create.blade.php
├── edit.blade.php
└── _form.blade.php
```

### 19. Index page role

`index.blade.php` contains:

- Add Page button
- table markup
- DataTable initialization
- no Laravel paginator

When using server-side DataTables, pagination should be handled by DataTables, not by Laravel paginator links in Blade.[cite:172] [cite:176]

### 20. DataTable initialization

```
@push('scripts')
<script>
    $(function () {
        $('#pagesTable').DataTable({
            processing: true,
            serverSide: true,
            ajax: '{{ route('admin.pages.datatable') }}',
            columns: [
```

```
                { data: 'DT_RowIndex', name: 'DT_RowIndex', orderable: false, searcha
                { data: 'title', name: 'title' },
                { data: 'slug', name: 'slug' },
                { data: 'meta_title', name: 'meta_title' },
                { data: 'status', name: 'status', orderable: false, searchable: false
                { data: 'is_published', name: 'is_published', orderable: false, sear
                { data: 'published_at', name: 'published_at' },
                { data: 'action', name: 'action', orderable: false, searchable: false
            ]
        });
    });
</script>
@endpush
```

## Shared Form Partial

### 21. Why partial was used

Create and Edit pages usually have the same fields, so the form was extracted into `_form.blade.php`. This keeps code smaller and easier to reuse in future modules.[cite:141] [cite:147]

### 22. Create page

```
@extends('admin.layouts.admin')

@section('title', 'Create Page')
@section('page_title', 'Create Page')

@section('content')
<form action="{{ route('admin.pages.store') }}" method="POST">
    @csrf
    @include('admin.pages._form', ['buttonText' => 'Create Page'])
</form>
@endsection
```

### 23. Edit page

```
@extends('admin.layouts.admin')

@section('title', 'Edit Page')
@section('page_title', 'Edit Page')

@section('content')
<form action="{{ route('admin.pages.update', $page->id) }}" method="POST">
    @csrf
    @method('PUT')
    @include('admin.pages._form', ['buttonText' => 'Update Page', 'page' => $page])
</form>
@endsection
```

### 24. Shared form logic

Use values like:

```
value="{{ old('title', $page->title ?? '') }}"
```

This allows the same partial to work for both create and edit forms.[cite:141] [cite:147]

## Custom Status Badges

Default badge styles were not giving enough visibility, so custom badge classes were added.

```
.badge-soft-success {
    background-color: #28a745 !important;
    color: #fff !important;
}

.badge-soft-danger {
    background-color: #dc3545 !important;
    color: #fff !important;
}

.badge-soft-primary {
    background-color: #007bff !important;
    color: #fff !important;
}

.badge-soft-warning {
    background-color: #ffc107 !important;
    color: #212529 !important;
}
```

## Common Errors Faced During Setup

### Route not defined

```
Route [admin.pages.datatable] not defined.
```

Fix:

- add datatable route in `web.php`
- confirm via `php artisan route:list`

### jQuery not defined

```
Uncaught ReferenceError: $ is not defined
```

Fix:

- load jQuery before DataTables and before `@stack('scripts')`.[cite:162] [cite:169]

### DataTables AJAX error

Fix:

- check AJAX route directly in browser
- fix namespace/import issue
- ensure route returns valid JSON instead of HTML error page.[cite:121] [cite:136]

### Pagination conflict

Fix:

- remove `{{ $pages->links() }}` from Blade
- do not mix Laravel paginator with DataTables pagination.[cite:172] [cite:176]

### Security Basics Used

- auth middleware for admin routes
- CSRF protection in forms
- form validation in store/update
- mass assignment protection through `$fillable`
- edit/delete only from protected admin area

These are essential Laravel protections for practical CRUD tasks.[cite:177]

### Reusable CRUD Pattern For New Tasks

The same structure can be reused for:

- blog posts
- categories
- products
- services
- testimonials
- leads
- demo videos
- FAQs

Reusable pattern:

1. create migration and model
2. create admin controller
3. add resource route + datatable route
4. create index/create/edit views
5. create `_form.blade.php`
6. add validation
7. implement server-side DataTable
8. add action buttons and badges

This makes the project easy to adapt during practical interview assignments.[cite:172] [cite:177]

## GitHub Preparation Checklist

Before pushing:

- remove `.env`
- keep `.env.example`
- do not upload `vendor/`
- do not upload `node_modules/`
- keep migrations
- keep README
- test login and admin CRUD once
- test DataTable listing once

## Suggested Future Modules

After Pages, good next modules would be:

- Page Sections
- Blog Posts
- Settings
- Leads
- Demo Videos

These fit well into the same Laravel admin pattern.[cite:177]